

# Taller de GitHub Actions: Implementación de Integración Continua y Protección de Ramas (DevSecOps)

## Integrantes:

Caetano Flores  
Jordan Guaman  
Mateo Medranda  
Anthony Morales  
Leonardo Narváez

## Docente:

David Galarza

07/05/2026





# El Escenario Base: La Puerta Abierta

sample\_dev\_sec\_opsPublic

main

1 Branch

0 Tags

Go to file

Add file

Code

MateoMedranda

first commit

9b0cf04 · 3 minutes ago

1 Commit

.github/workflows

first commit

3 minutes ago

\_tests\_

first commit

3 minutes ago

dist

first commit

3 minutes ago

public

first commit

3 minutes ago

src

first commit

3 minutes ago

.gitignore

first commit

3 minutes ago

package-lock.json

first commit

3 minutes ago

package.json

first commit

3 minutes ago

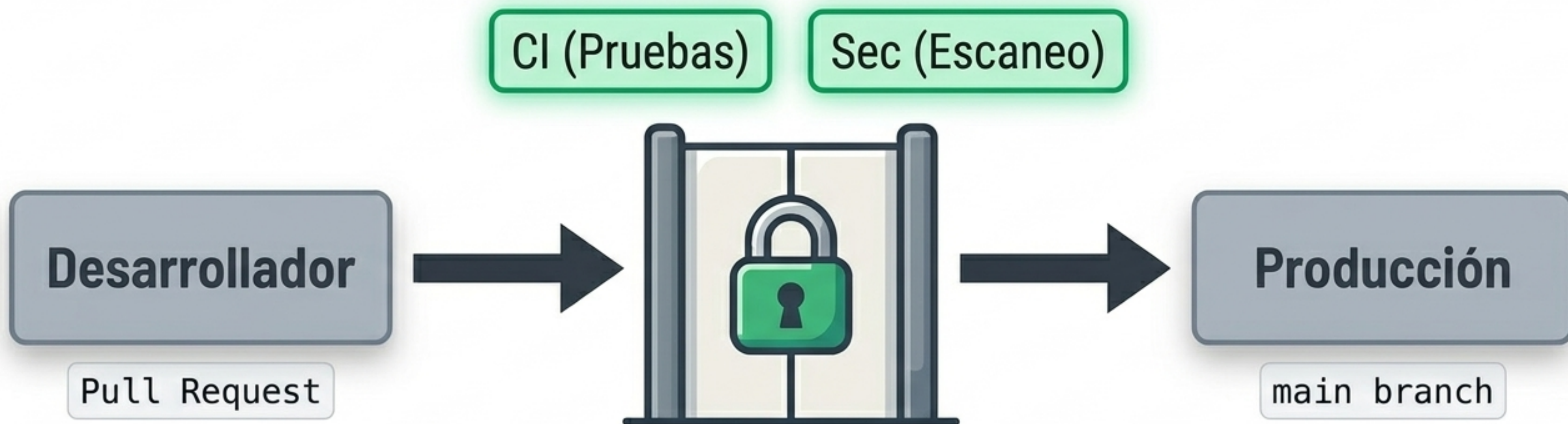
## El Problema Central

Nuestro repositorio ejecuta pipelines de GitHub Actions (`devsecops.yml`), pero no hay reglas que exijan su cumplimiento.

|                                                                                       |                                                                                                                                  |
|---------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------|
|  | <b>Fallo Permitido</b> <p>La fusión de código hacia main es permitida incluso si los análisis de seguridad o pruebas fallan.</p> |
|  | <b>Riesgo de Producción</b> <p>Código vulnerable o roto puede integrarse sin restricciones, rompiendo la aplicación en vivo.</p> |



# El Objetivo: Políticas de Zero Trust en el Repositorio



Transformar el repositorio en una fábrica segura. Ninguna línea de código entra a producción a menos que apruebe explícitamente la Integración Continua (CI) y los escaneos de seguridad automatizados.



# La Intervención: Configurando Branch Protection

## Settings Panel

**1 Ruta de Navegación**      Settings > Branches > Branch protection rules

**2 Regla Principal**      ☒ Require status checks to pass before merging

**3 Checks Obligatorios**      build\_and\_test      security\_scan

**Resultado Esperado:** El sistema bloqueará físicamente el botón de Merge hasta que los pipelines obligatorios finalicen con éxito.



# Prueba de Fuego: Inyectando Código Vulnerable

## Add a title

Taller Grupo # Fixes Loadsh

## Add a description

Taller Grupo # Fixes Loadsh

Create pull request

package.json

```
{  
  "dependenct": "9.1.0",  
  "dependence": "9.1.3",  
  "lodash": "4.17.19",  
  "rependences": "3.1.0",  
  "github": "^8.0.0"  
}
```

## Vulnerabilidad Intencional

Para comprobar la eficacia de nuestras nuevas reglas, abrimos un Pull Request que incluye una versión desactualizada de la dependencia lodash, la cual contiene vulnerabilidades conocidas.



# El Pipeline Reacciona: Vulnerabilidad Detectada

## Some checks were not successful

[Hide all checks](#)

1 successful, 1 failing, and 1 skipped checks

1 successful, 1 failing, and 1 skipped checks

|   |                                                                                                                                                      |                         |
|---|------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------|
| ✗ |  DevSecOps Demo Pipeline / security_scan (push) Failing after 15s   | <a href="#">Details</a> |
| ✓ |  DevSecOps Demo Pipeline / build_and_test (push) Successful in 12s | <a href="#">Details</a> |
| ⊘ |  DevSecOps Demo Pipeline / deploy (push) Skipped                  | <a href="#">Details</a> |





# Integración Denegada: La Regla en Acción



**Some checks were not successful**

Merge pull request

Merging is blocked due to failing merge requirements

## ¡Bloqueo Exitoso!

Gracias a la protección de rama, GitHub reconoce el fallo en el pipeline de seguridad.

El botón se deshabilita automáticamente, protegiendo la rama principal. El desarrollador no puede eludir esta restricción.



# La Corrección: Actualizando Dependencias Inseguras

| package.json |                     |
|--------------|---------------------|
|              |                     |
| -            | "lodash": "4.17.19" |
| +            | "lodash": "4.17.21" |

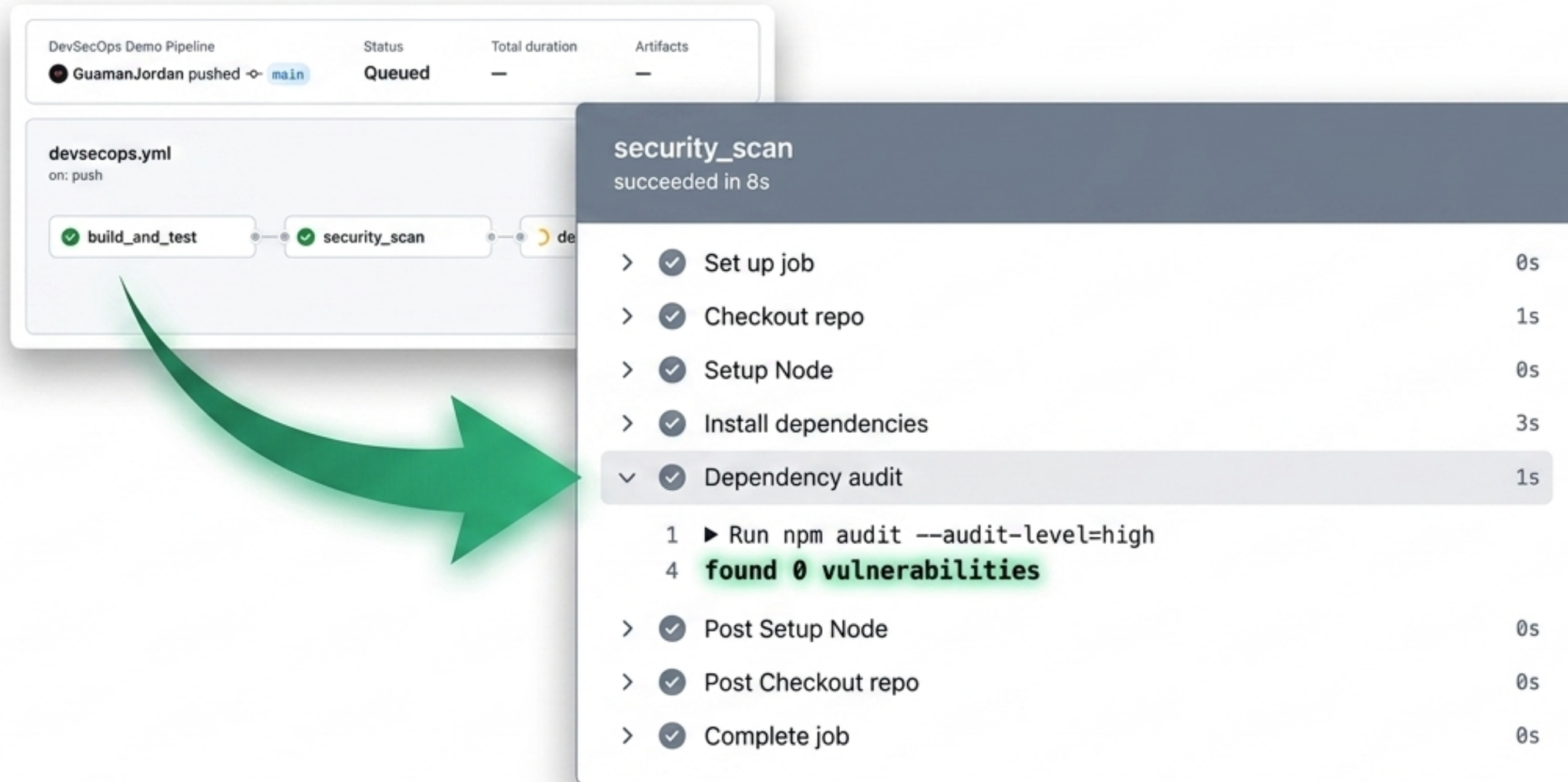
Diagram illustrating the correction of insecure dependencies in a `package.json` file:

- The vulnerable version `"4.17.19"` is removed (indicated by a red background and the label "Versión vulnerable removida").
- The secure version `"4.17.21"` is injected (indicated by a green background and the label "Versión segura inyectada").

**Acción:** El desarrollador realiza un nuevo commit para parchear la vulnerabilidad. Al hacer push, el ciclo de DevSecOps se reinicia automáticamente.



# Re-evaluación: Auditoría de Seguridad Limpia



The image shows a GitHub Actions workflow interface. At the top, a pipeline named 'DevSecOps Demo Pipeline' is shown with a status of 'Queued' and a trigger of 'GuamanJordan pushed to main'. Below this, a job named 'devsecops.yml' is shown with a trigger of 'on: push'. The job contains three steps: 'build\_and\_test' (green checkmark), 'security\_scan' (green checkmark), and 'deploy' (yellow circle). A large green arrow points from the 'security\_scan' step in the job list to a detailed view of that step. The detailed view shows the step name 'security\_scan' and its status 'succeeded in 8s'. Below this, a list of sub-steps is shown with their durations: 'Set up job' (0s), 'Checkout repo' (1s), 'Setup Node' (0s), 'Install dependencies' (3s), 'Dependency audit' (1s), 'Post Setup Node' (0s), 'Post Checkout repo' (0s), and 'Complete job' (0s). The 'Dependency audit' step is expanded, showing a command 'Run npm audit --audit-level=high' and the output 'found 0 vulnerabilities'.

DevSecOps Demo Pipeline

GuamanJordan pushed to main

Status: Queued

Total duration: —

Artifacts: —

devsecops.yml

on: push

build\_and\_test

security\_scan

security\_scan

succeeded in 8s

- > ✓ Set up job 0s
- > ✓ Checkout repo 1s
- > ✓ Setup Node 0s
- > ✓ Install dependencies 3s
- ✓ Dependency audit 1s
  - 1 ▶ Run npm audit --audit-level=high
  - 4 **found 0 vulnerabilities**
- > ✓ Post Setup Node 0s
- > ✓ Post Checkout repo 0s
- > ✓ Complete job 0s

El nuevo código se analiza y valida que la amenaza fue mitigada.



# Vía Libre: El Pipeline DevSecOps en Verde

 **Status: Success**  
Total duration: 1m 10s



¡Éxito! Todos los trabajos configurados han pasado exitosamente. El código ahora cuenta con la certificación automatizada de integridad estructural y seguridad.



# Fusión Autorizada (Merge Allowed)



All checks have passed



build\_and\_test



security\_scan

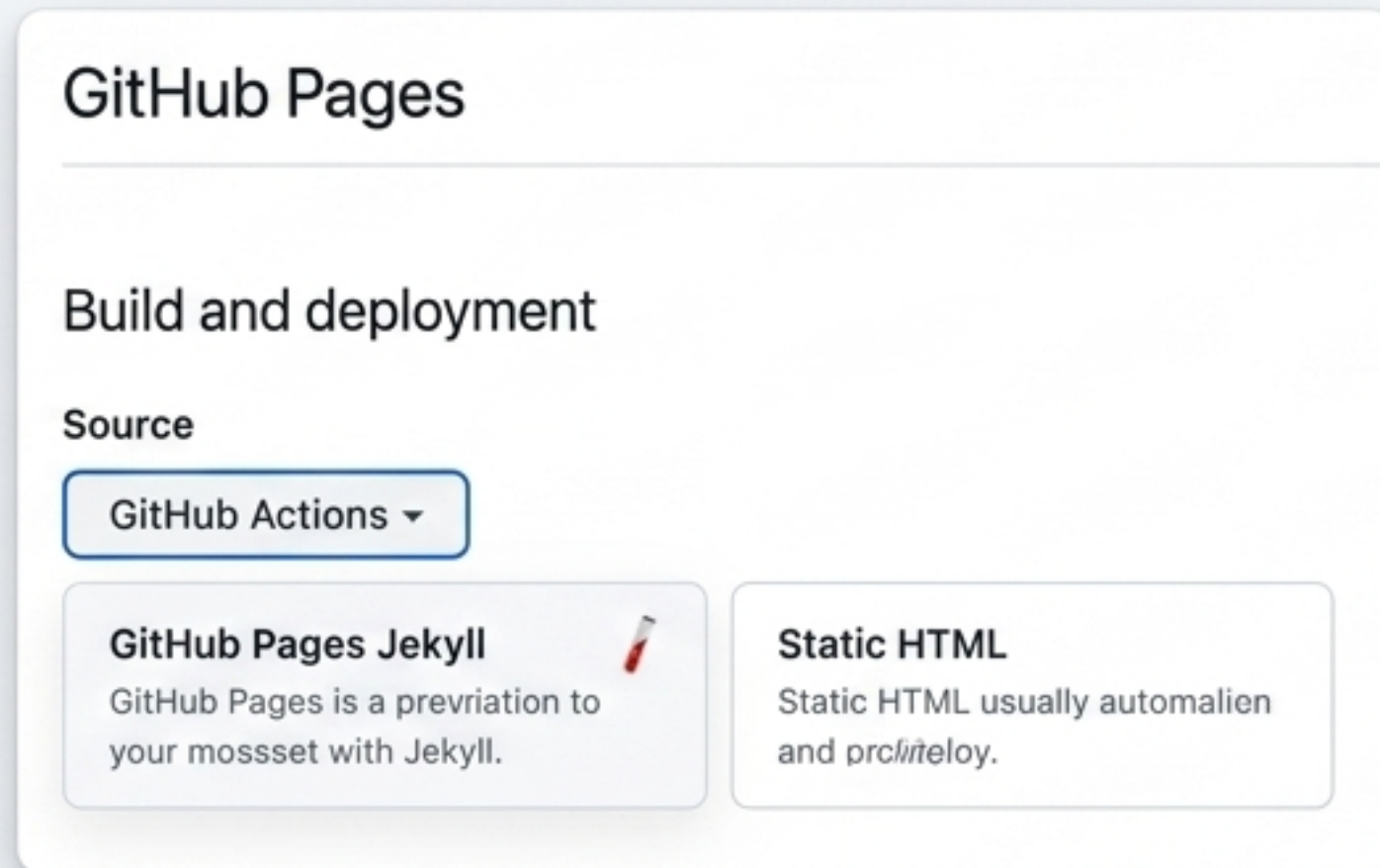
Merge pull request

- Branch Protection satisfecha.
- Barrera de seguridad desbloqueada.
- El código seguro se integra a la rama main.



# Continuous Deployment (CD): Despliegue Automático Seguro

## The Mechanism



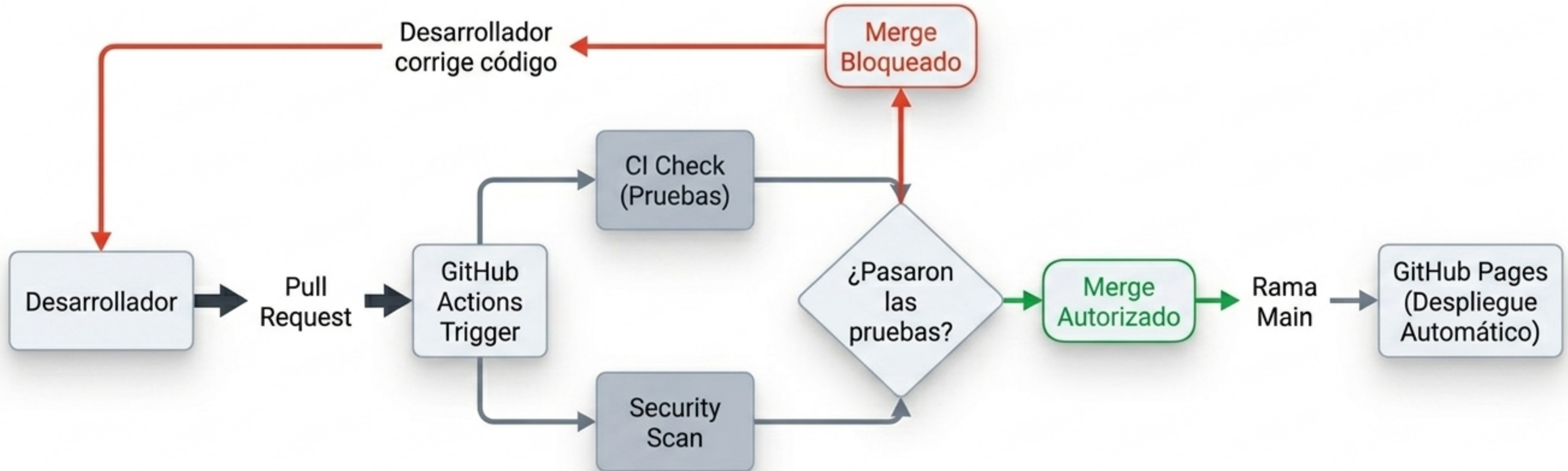
Una vez fusionado en main, el último paso del pipeline empaqueta la aplicación automáticamente.

## The Result





# El Flujo DevSecOps Implementado (Síntesis)



Al combinar GitHub Actions con Branch Protection, transformamos un repositorio estático en una fábrica de software segura. La seguridad no es una revisión manual al final; es una barrera automatizada por diseño.